

DE-04

Schema Drift Detector

Snapshots SQLite/Postgres schemas daily, diffs against previous snapshot, and uses an LLM to rate each change and generate mitigation notifications.

Diff + LLM Rating

SQLite / Postgres

Breaking Detection

Auto Mitigation

USE CASES

ID	Use Case	Description	Type
UC-01	Detect breaking changes	Catches dropped columns, renamed tables, or changed data types that silently break downstream ETL pipelines and APIs.	Breaking
UC-02	Track additive growth	Monitors new tables, optional columns, or added indexes so teams know the schema is evolving without risk.	Additive
UC-03	Auto-generate mitigations	LLM drafts actionable mitigation steps (e.g., "update ORM model", "add NULL check") for every detected change.	AI-powered
UC-04	Daily snapshot audit	Maintains a version history of JSON snapshots so teams can diff any two points in time for compliance and rollback.	Audit
UC-05	Slack / email alert	Sends structured notifications to on-call engineers only when LLM rates a change as breaking — reduces alert fatigue.	Alert
UC-06	Multi-database support	Supports SQLite (dev/test) and Postgres (prod), enabling the same drift detection pipeline across every environment.	Cross-DB

AI TOOLS USED — ROLES & CONTRIBUTIONS

Claude (Anthropic)

Core LLM Engine — Impact Rating & Mitigation

- Classifies each schema diff as additive or breaking with confidence score
- Writes human-readable mitigation steps per detected change
- Drafts Slack/email alert body with full change summary
- Explains downstream impact (ORM fields, API contracts)

ChatGPT (OpenAI)

Architecture Planning & Prompt Design

- Brainstormed overall pipeline architecture (snapshot → diff → rate)
- Designed the JSON schema format for storing snapshots
- Drafted and iterated on the LLM classification prompt
- Identified edge cases: NULL constraints, enum changes, FK drops

Cursor (AI IDE)

Code Generation & Refactoring

- Generated SQLite/Postgres schema introspection queries
- Wrote the JSON diff engine (added/removed/modified detection)
- Scaffolded the daily cron scheduler and file storage logic
- Refactored notification module for readability and testability

Antigravity

UI / Dashboard Scaffolding

- Scaffolded the web dashboard for viewing snapshot history
- Generated the side-by-side schema diff viewer component
- Created the change timeline and severity color-coding UI
- Built the notification log table with filter and search

WHERE AI WAS USED — PIPELINE STAGES

STAGE 1 — SCHEMA SNAPSHOT

Introspects SQLite/Postgres and serialises tables, columns, types, constraints to JSON

Cursor: introspection queries

ChatGPT: JSON schema design

STAGE 2 — DIFF ENGINE

Compares today's JSON snapshot against yesterday's; produces a structured diff object

Cursor: diff logic

ChatGPT: edge case enum

STAGE 3 — LLM IMPACT RATING

Sends each diff item to Claude; receives breaking/additive rating + confidence + mitigation

Claude: rating & mitigation

ChatGPT: prompt engineering

STAGE 4 — NOTIFICATION GENERATION

Claude drafts Slack/email alert body; only breaking changes trigger a notification

Claude: notification copy

Cursor: dispatcher integration

STAGE 5 — DASHBOARD / HISTORY VIEW

Web UI shows snapshot history, diff viewer, severity heatmap, and notification log

Antigravity: full dashboard UI

Cursor: API endpoints

TEST CASES

TC ID	Scenario	Input (Schema Change)	Expected LLM Rating	Expected Result
TC-01	No schema change	Snapshots identical	—	✓ No alert sent
TC-02	New optional column added	ALTER TABLE orders ADD COLUMN notes TEXT;	Additive	✓ Log entry, no alert
TC-03	Column dropped	ALTER TABLE users DROP COLUMN email;	Breaking	✓ Alert + mitigation sent
TC-04	Column type changed	price: REAL -> INTEGER	Breaking	✓ Alert + migration advice
TC-05	Table renamed	orders -> purchase_orders	Breaking	✓ Old=removed, New=added flags
TC-06	New table added	CREATE TABLE audit_log (...);	Additive	✓ Log entry, no alert
TC-07	NOT NULL constraint added	phone TEXT -> phone TEXT NOT NULL	Breaking (data-dep)	✓ Alert with NULL-check advice

TC-08	Foreign key dropped	DROP CONSTRAINT fk_user_id	Breaking	✓ Integrity risk alert
TC-09	Index added (perf only)	CREATE INDEX idx_email ON users(email);	Additive	✓ No alert, snapshot updated
TC-10	LLM API timeout	Network failure mid-rating	—	■ Retry 3x then error log
TC-11	Snapshot file corrupted	Malformed JSON on disk	—	■ Fallback to day-2 snapshot
TC-12	Multiple breaking changes	3 columns dropped + type change	Breaking x4	✓ Single batched alert, all items